Take-Home Assignment — Driver Delivery App

Mobile Engineer (Flutter) · TransVirtual · Candidate brief

Overview

Build a small, working slice of a driver's delivery app: a Flutter app for viewing today's delivery stops and updating their status — designed offline-first, because our drivers work in depots and dead zones. We care most that it runs end-to-end as a workable demo on an emulator or simulator, and that your offline design survives a restart.

What we provide

- deliveries.csv — 30 delivery stops with columns: reference, customer_name, address, latitude, longitude, status, time_window.

That is all. Local storage choice, sync design, state management, and every other tooling decision are yours — deliberately. They are part of what we evaluate.

Requirements

1. Data: bundle the provided deliveries.csv as an asset and load it into local storage of your choice (sqflite, Drift, Hive, Isar — your call).
2. Stop list screen: today's stops with reference, customer, address, and status, filterable by status.
3. Stop detail and status update: mark a stop completed, or failed with a required failure reason. Lifecycle: pending → completed | failed; completed and failed are terminal — invalid updates must be rejected in logic, not just hidden in the UI.
4. Offline-first: every update persists locally and survives an app restart. Maintain a visible pending-sync queue in the UI.
5. Sync: include an in-app connectivity toggle. While “offline”, updates accumulate in the queue; when switched “online”, the queue flushes to a mock sync target (a stub server, a logged endpoint, or an in-app fake — your design) and the UI reflects synced state.
6. State management: your choice — be ready to defend it.
7. Tests: at least two meaningful tests — we suggest one on the status lifecycle and one on the offline queue behaviour.
8. Workable demo: the app runs on an Android emulator or iOS simulator with documented commands (flutter run is fine). Demo flow we'll follow: go offline → update several stops → restart the app → go online → watch the queue sync.

Ground rules

- Timebox: 4–6 hours of effort, due within 2 days of receiving this brief (a 3rd day is available on request). Stop at the timebox and write down what you would do next — unfinished-but-well-reasoned beats gold-plated.
- AI tools (Claude Code, Copilot, Cursor, etc.) are allowed and encouraged — we work AI-first. Include a short AI-usage note: which tools, what was generated vs. hand-written, and at least one thing the AI got wrong and how you caught it.
- Use any packages you like. Skip authentication, real backend integration, push notifications, and anything not listed in the requirements.
- You will demo this live (about 5 minutes) at the start of your final interview and extend it during the session — build something you can talk about.

What to submit

- A Git repository link (or zip) with the full source.
- README.md: how to run the demo (the exact commands and target emulator/simulator), key decisions — storage, sync, and state-management choices and why — and your “what I'd do next” list.
- Your AI-usage note (in the README or separate).

How we evaluate

- The demo is the deliverable: if we cannot run your app with the documented commands, we will ask you once to fix the setup; if it still doesn't run, we can't progress the application. The demo matters more than any individual feature.
- We look at: offline and sync design · correctness of the status lifecycle · code structure and state management · setup and tooling choices · testing · the quality of decisions explained in your README · judgment in how you used AI tools.
- In the final interview you'll demo the app, defend your design as we scale the constraints (thousands of stops, photo/signature payloads, conflict resolution, battery), and discuss how you'd help junior engineers pick up Flutter.